COURSE OUTLINE

Department:	Faculty of Computing	**Knowledge Group:**	Programming Core	
Programme:	Software Engineering	**Class:**	BESE15-AB	
Course code:	CS-212	**Academic Session/Semester:**		Spring 2025
Course name:	Object Oriented Programming	**Pre/co requisite (course name and code, if applicable):**		CS-110 FoCP
Credit hours:	3+1			

Course Synopsis	This course will introduce the Object-Oriented (OO) philosophy to software development, which is a modern and powerful approach. Today, many programming languages support the OO concepts. However, during this course, we'll majorly use the Java programming language. The objective is to make the students understand the benefits of using OO techniques over procedural programming practices, and thereby motivating them to use OO concepts in software development. Further, the students will learn some powerful features of the Java programming language.			
Course Learning Outcomes (CLOs)	1. Understand core OOP concepts such as classes, objects, encapsulation, inheritance, and polymorphism. 2. Evaluate complex programming problems using object-oriented principles analysing solutions for efficiency and scalability. 3. Develop solutions to real-world problems by applying appropriate object-oriented techniques. 4. Build programs using the latest IDEs as per standard practices applicable to the software industry. 5. Demonstrate commitment to professional ethics by following engineering norms applicable to the software industry.			
Course Schedule	Tuesday 09:00-10:50, Wednesday 09:00-10:50 and Friday 09:00-10:50 Lab (Mondays 10:00-12:50 & 14:00-17:00)			
Course lecturer	**Name**	**Office**	**Contact#**	**E-mail**
	Mehwish Kiran	A-208		mehwish.kiran@seecs.edu.pk

Mapping of the Course Learning Outcomes (CLO) to the Programme Learning Outcomes (PLO), Teaching & Learning (T&L) methods and Assessment methods:

No.	Course Learning Outcomes	PLO (SE)	PLO (CS)	BT Level	Teaching & Learning Methods	Assessment Methods
CLO 1	**Understand** core OOP concepts such as classes, objects, encapsulation, inheritance, and polymorphism.	PLO-1	PLO-1	C-2 (Understand)	Active learning, Cooperative Learning, Blended Learning	Assignment Quiz MSE ESE
CLO 2	**Evaluate** complex programming problems using object-oriented principles analysing solutions for efficiency and scalability.	PLO-2	PLO-3	C-5 (Evaluate)	Active learning, Cooperative Learning, Blended Learning	Assignment Quiz MSE ESE Labs
CLO 3	**Develop** solutions to real-world problems by applying appropriate object-oriented techniques.	PLO-3	PLO-4	C-6 (Create)	Active learning, Cooperative Learning, Blended Learning	MSE ESE Assignments Project

						Labs
CLO 4	**Build** programs using the latest IDEs as per standard practices applicable to the software industry.	PLO-5	PLO-5	P-3 (Guided Response)	Active learning, Cooperative Learning, Blended Learning	Project Labs
CLO 5	**Demonstrate** commitment to professional ethics by following engineering norms applicable to the software industry.	PLO-7	PLO-9	A-3 (Valuing)	Active learning, Cooperative Learning, Blended Learning	Project Labs

Details on Innovative T&L practices:

No.	Type	Implementation
1.	Active learning	Conducted through in-class or lab activities.
2.	Cooperative learning	Conducted through design project. Students in a team of five will be given a design project that requires software engineering process design solutions. Compliance to the design specifications need to be given in the form of written reports.
3.	Blended learning	Conducted through Learning Management System (LMS) of NUST. All information as well as materials related to teaching and learning activities will be shared with class through this system. Some for formative assessments will be also conducted using this system.

Weekly Schedule:

Week 1	OOP / Java Basics • Introduction to OOP and core OOP principles • Introduction to Java Programming Language • Compiling & Executing Simple Java Programs • Data types & Operators, Input / Output • Decision • Loops and iterations
Week 2	Functions Arrays and strings
Week 3 Week 4	Classes and Objects • Basic Concepts • Encapsulation & Data Hiding • Access modifiers • Constructors & Destructors • Setters & Getters • Copy constructor • Default and No-Argument Constructors • Method (Function) Overloading • Static Class Members • this Reference • Understanding UML Class Diagrams
Week 5 Week 6	Inheritance In Java • Super classes and sub classes

	• Protected members • Method (Function) Overriding • Constructors in subclasses / constructors chaining
Week 7	Polymorphism • Method overloading vs. method overriding • Dynamic method dispatch (Runtime Polymorphism) • Final Methods and Classes
Week 8	Abstraction - Interfaces & Abstract Classes • Difference between abstraction and encapsulation • Abstract classes and methods (abstract keyword) • Defining and implementing interfaces (implements) • Interface vs. Abstract class
Week 9	**Mid-Semester Break**
Week 10	Composition vs. Inheritance • Understanding Composition (HAS-A relationship) • Why Composition is often preferred over Inheritance • Example: Car-Class with Engine as a separate component
Week 11	Object Relationships & UML Diagrams • Association, Aggregation, and Composition • Understanding UML Class Diagrams • Implementing UML-based class relationships in Java
Week 12	Exception Handling • Types of exceptions: Checked vs. Unchecked • try-catch-finally blocks • Creating and using custom exceptions
Week 13	The SOLID Principles (Overview) • Single Responsibility Principle (SRP) • Open-Closed Principle (OCP) • Dependency Inversion Principle (DIP) • How these principles make code more maintainable
Week 14	File Handling & Object Serialization • Reading and writing files (FileReader, FileWriter) • Object serialization (ObjectOutputStream, ObjectInputStream) • Saving and retrieving objects in a file
Week 15	OOP Case Study - Building a Mini Project • Students will choose a project (e.g., Library Management, Student Records System) • Apply Encapsulation, Inheritance, Polymorphism, and Abstraction
Week 16	Code Review & Optimization • Debugging & refactoring OOP code • Best practices in OOP • Performance considerations in Java OOP
Week 17	Project Presentations / Revisions
Week 18	**End Semester Break**

Lab Experiments (if applicable):

Lab 1	Java Basics [Data types and Operators]
Lab 2	Flow Control [Loops and Decisions]
Lab 3	Composite data types [Arrays & Strings]
Lab 4	Classes and Objects
Lab 5	Constructors
Lab 6	Inheritance
Lab 7	Function Overloading
Lab 8	Function Overriding
Week 9	**Mid-Semester Break**
Lab 9	Polymorphism
Lab 10	Interfaces and Abstract Classes
Lab 11	Exception Handling
Lab 12	GUI
Lab 13	Event Handling
Lab 14	I/O and File Handling
Lab 15	Open-Ended Lab
Lab 16	Advanced Topics
Week 18	**End Semester Break**

Assessment Methods:

Assessment		Percentage
Theory: 75%		
1	Quizzes (10-15%)	15%
2	Assignments (5-10%)	10%
3	Project	10%
4	Mid-Term Exam (25-35%)	25%
5	End-Semester Exam (40-50%)	40%
Labs:25%		
1	Labs Tasks	70%
2	Final Lab	20%
3	Semester Project	10%
Total:		100

Learning resources:

Textbook:

1. Java – How to Program (Early Objects), Paul Deitel and Harvey Deitel, 10th Edition, ISBN 978-1- 292-01819-5, Pearson Education, 2015
2. Bruce Eckel, Thinking in Java, Fourth Edition, ISBN-13: 978-0131872486, 25th March, 2006. (available online)

Reference Book:

1. Computing Concepts with Java Essentials, Cay Horstmann, 3rd Edition, ISBN 0-471-24371-x, 2003, John Wiley & Sons.
2. Object Oriented Programming in C++, Robert Lafore, 4th Edition, ISBN-10: 0672323087 | ISBN-13: 978-0672323089, 2001

Grading Policy:

Quiz Policy:

The quizzes will be unannounced / announced and normally last for ten minutes. The question framed is to test the concepts involved in last few lectures. Number of quizzes that will be used for evaluation is at the instructor's discretion.

Project Policy:

Students will be required to develop a project during the course which should be completed towards the end of the semester. They will be graded based on project deliverables and presentation at the end. Students will work in a group/team for projects. A group of 3 students is recommended. At most 4 students are allowed.

Assignment Policy:

In order to develop comprehensive understanding of the subject, assignments will be given. Late assignments will not be accepted / graded. All assignments will count towards the total (No 'best-of' policy). The students are advised to do the assignment themselves. Copying of assignments is highly discouraged and violations will be dealt with severely by referring any occurrences to the disciplinary committee. The questions in the assignment are meant to be challenging to give students confidence and extensive knowledge about the subject matter and enable them to prepare for the exams.

Class participation:

The students are encouraged to participate in class by actively taking part in asking questions from the instructor, sharing his/her thoughts about the topic under discussion, replying to instructor questions, contribute in project presentation and demo. The class participation will be recorded by the instructor and 2% of project marks are assigned to student class participation.

Plagiarism:

SEECS maintains a zero-tolerance policy towards plagiarism. While collaboration in this course is highly encouraged, you must ensure that you do not claim other people's work/ ideas as your own. Plagiarism occurs when the words, ideas, assertions, theories, figures, images, programming codes of others are presented as your own work. You must cite and acknowledge all sources of information in your assignments. Failing to comply with the SEECS plagiarism policy will lead to strict penalties including zero marks in assignments and referral to the academic coordination office for disciplinary action.

